Session 6: File I/O & Data Persistence

Take-Home Practice Problems

Introduction

Welcome to Session 06 Practice!

These problems will help you master the key concepts from Session 06:
Topics Covered:

- File I/O: reading, writing, appending
- File modes: 'r', 'w', 'a'
- Working with structured data (CSV format)
- Error handling with files
- Data persistence across program runs

What Makes These Problems Better:

- ✔ **Clear requirements** - Each function has specific inputs and outputs
- ✔ **Built-in tests** - Run your code and see if it passes!
- ✔ **Formatted output** - All printing is handled for you
- ✔ **Focus on logic** - Just write the function bodies

How to Complete the Problems

1. Open the `SESSION_06_TAKE_HOME_PROBLEMS` folder
2. You'll find 4 Python files, one for each question
3. Each file has:
    - Function templates with clear documentation
    - TODO comments showing where to write your code
    - Test code that runs automatically
    - Expected output examples
4. Complete each function by replacing `pass` with your code
5. Run the file to test your implementation
6. Don't modify the test code at the bottom!

Question 1: File Logger

File: `question1_file_logger.py`

Topics: Writing files, appending, reading files

What You'll Do:

Implement three functions to work with log files:

1. `create_log_file()` - Create a new log file with header
2. `add_log_entry()` - Append a timestamped entry to the log
3. `read_recent_logs()` - Read and return the last N log entries

Key Skills:

- Using 'w' mode to create files
- Using 'a' mode to append without overwriting
- Reading files line by line
- Slicing to get recent entries

Expected Behavior:

```
# Create log
create_log_file("activity.log")
# Creates file with header: "=== Activity Log ==="

# Add entries
add_log_entry("activity.log", "User logged in")
add_log_entry("activity.log", "File uploaded")
add_log_entry("activity.log", "User logged out")

# Read recent
recent = read_recent_logs("activity.log", 2)
# Returns: ["[2025-01-15] File uploaded", "[2025-01-15] User logged out
    "]
```

Hints:

- Use `with open(filename, 'w')` for creating
- Use `with open(filename, 'a')` for appending
- Use list slicing `[-n:]` to get last n items
- Remember to add \n newlines when writing!

Question 2: CSV Contact Book

File: `question2_csv_contacts.py`

Topics: CSV format, dictionaries, structured data

What You'll Do:

Implement functions to manage contacts in CSV format:

1. `save_contacts()` - Save dictionary of contacts to CSV
2. `load_contacts()` - Load CSV file into dictionary
3. `add_contact()` - Add new contact and save to file
4. `search_contact()` - Search for a contact by name

Key Skills:

- Writing CSV with headers
- Using `split(",")` to parse CSV lines
- Converting between dictionaries and CSV format
- Handling file operations for CRUD operations

CSV Format:

```
name,phone,email
Alice,555-1234,alice@email.com
Bob,555-5678,bob@email.com
```

Data Structure:

```
contacts = {
    "Alice": {"phone": "555-1234", "email": "alice@email.com"},
    "Bob": {"phone": "555-5678", "email": "bob@email.com"}
}
```

Hints:

- First line should be: "name,phone,email\n"
- Use `line.strip().split(",")` to parse each line
- Skip the header line when reading: `lines[1:]`
- Check if contact exists before adding

Question 3: Student Scores Tracker

File: `question3_scores_tracker.py`

Topics: Lists in CSV, data persistence, error handling

What You'll Do:

Implement a student scores tracking system with persistence:

1. `save_scores()` - Save {name: [scores]} to CSV
2. `load_scores()` - Load CSV into {name: [scores]}
3. `add_score()` - Add score to student (updates file)
4. `get_statistics()` - Calculate min, max, average for student

Key Skills:

- Storing lists as comma-separated values
- Converting string of numbers to list of ints
- Error handling with `try/except`
- Checking if file exists before reading

CSV Format:

```
name,scores
Alice,85,92,78,88
Bob,90,88,95
Charlie,75,80,82,78,85
```

Data Structure:

```
scores = {
    "Alice": [85, 92, 78, 88],
    "Bob": [90, 88, 95],
    "Charlie": [75, 80, 82, 78, 85]
}
```

Hints:

- Save scores as: `",".join(map(str, scores_list))`
- Load scores as: `list(map(int, scores_str.split(",")))`
- Handle FileNotFoundError with try/except
- Return empty dict if file doesn't exist

Question 4: Configuration File Manager

File: `question4_config_manager.py`

Topics: Key-value pairs, file persistence, default values

What You'll Do:

Implement a configuration manager for application settings:

1. `load_config()` - Load config from file (with defaults)
2. `save_config()` - Save config dictionary to file
3. `get_setting()` - Get setting value (with default)
4. `update_setting()` - Update setting and save to file

Key Skills:

- Key-value format: `key=value`
- Default values when file doesn't exist
- Type conversion for different setting types
- Robust error handling

Config File Format:

```
theme=dark
font_size=14
auto_save=True
language=Python
```

Data Structure:

```
config = {
    "theme": "dark",
    "font_size": "14",
    "auto_save": "True",
    "language": "Python"
}
```

Hints:

- Parse lines with: `key, value = line.strip().split("=")`
- Provide default config if file doesn't exist
- Use `config.get(key, default)` for safe access
- Save after every update for persistence

Bonus Challenge: Data Migration Tool

If you finish all problems and want extra practice:

The Challenge:

Create a tool that converts between different data formats:

- Read from one CSV format
- Transform the data
- Write to a different CSV format

Example:

```
# Input: students.csv
name,test1,test2,test3
Alice,85,92,78
Bob,90,88,95

# Output: report.csv
name,average,grade
Alice,85.0,B
Bob,91.0,A
```

Skills Used:

- Reading and parsing CSV
- Data transformation
- Writing new CSV format
- Error handling for both files

Testing Your Solutions

Each file includes test code at the bottom. To test:

1. Complete the function implementations
2. Save the file
3. Run: `python question1_file_logger.py`
4. Check the output matches expected results
5. Verify files are created/modified correctly

Success Criteria

You've mastered Session 06 when you can:

- Create, read, write, and append to files confidently
- Parse and create CSV format data
- Handle file errors gracefully
- Convert between Python data structures and file formats
- Build programs that persist data across runs

Common Mistakes to Avoid

- **Forgetting newlines:** Always add \n when writing!
- **Not stripping:** Use `strip()` before `split()`
- **Type conversion:** Everything from files is a string!
- **File modes:** 'w' overwrites, 'a' appends
- **Closing files:** Always use `with` statement

Need Help?

- Review the session slides and examples
- Check the lecture notes for reference
- Test small parts of your code separately
- Print intermediate values to debug
- Ask questions in office hours!